GG Power · Backend

Kế hoạch nâng cấp backend

Chạy nhiều instance và mở rộng quy mô — mục 4.4 của PROGRESS.md

Phiên bản 1.0 · 11/10/2026 · Nhánh feature/contract-and-theme

Tài liệu này vừa giải thích **vì sao** cần nâng cấp, vừa là **bản kế hoạch từng bước** để Claude (hoặc lập trình viên) làm theo đúng trình tự. Mỗi bước có mục tiêu, việc cần làm, file liên quan và tiêu chí "xong".

1. Cách dùng tài liệu này (quy tắc cho Claude)

1. Làm **đúng thứ tự** A → B → C, trong mỗi giai đoạn theo số bước. Không nhảy bước trừ khi người dùng yêu cầu.
2. **Mỗi bước một commit**, message dạng `feat(backend): A4 tổng hợp tăng dần` (kết thúc bằng dòng Co-Authored-By theo quy ước của repo).
3. Trước khi commit: `dotnet test` **xanh** (kể cả `ContractTests`); nếu bước có chạm app thì chạy cả cổng kiểm tra của app.
4. Không đổi API công khai nếu bước không yêu cầu. Nếu buộc phải đổi: sửa `contract/openapi.yaml` → thêm mục `contract/CHANGES.md` → code + test.
5. Tới điểm có dòng **"Cần người dùng quyết"**: hỏi bằng hộp chọn (popup) với phương án đề xuất đứng đầu, ghi quyết định vào mục 2 của `PROGRESS.md` rồi mới làm.
6. Theo dõi tiến độ trong **PROGRESS.md mục 4.4** (bảng trạng thái từng bước). File Word này là đặc tả, không sửa khi làm; đổi kế hoạch thì xuất bản phiên bản mới.
7. **Không deploy production**, không đổi máy chủ thật. Mọi việc chạy trên máy dev và Docker cục bộ; hướng dẫn triển khai được viết vào `backend/docs/`.
8. Sau mỗi giai đoạn chạy lại bài thử tải (bước A0) và ghi kết quả vào `backend/docs/LOADTEST.md` để so sánh với số nền.

Môi trường test hiện có: PostgreSQL trong Docker (`ggpower-pg-test`, cổng 5433), chạy test với `DOTNET_ROLL_FORWARD=Major` (xem PROGRESS.md mục 5).

2. Hiện trạng: vì sao chưa chạy được nhiều instance

Backend hiện là **một tiến trình làm tất cả**: nhận MQTT, giữ trạng thái thiết bị trong RAM, phát WebSocket, điều phối lệnh ghi cấu hình và chạy các job định kỳ. Bật instance thứ hai sẽ hỏng như sau:

Thành phần chỉ có ở một máy	Hậu quả khi có 2 instance
MQTT đăng ký `solar/{tenant}/inverter/+/up/#`, `CleanSession`; Mosquitto bật `use_username_as_clientid` nên client id = tên đăng nhập MQTT	Hai instance dùng chung tài khoản MQTT sẽ **đá nhau ra** liên tục; dùng hai tài khoản thì cả hai nhận mọi bản tin → cảnh báo, thông báo đẩy bị nhân đôi
`LiveStateCache` là nguồn số liệu realtime của `/overview`, `/devices`, `/telemetry`	Nếu chia bản tin cho hai máy, mỗi máy chỉ biết một nửa thiết bị; request rơi vào máy kia thấy thiết bị "offline"
`RealtimeHub` (WebSocket) trong RAM	App nối vào máy A chỉ nhận frame của thiết bị mà máy A xử lý
`RealtimeTickets` trong RAM	Vé tạo ở A, WebSocket nối vào B → bị từ chối
`CommandService`: `_acks`, `_running`, `_lastCommand` trong RAM	Ack của inverter có thể về máy không chờ nó → lệnh báo "không rõ"; giới hạn tốc độ và khoá "đang ghi" mất tác dụng
Job định kỳ: tổng hợp 30 s, `LiveMonitor` 10 s, báo cáo ngày mỗi phút, nạp lại TOU 00:00, dọn dữ liệu 6 h	Chạy ở mọi máy: **báo cáo ngày gửi 2 lần**, cảnh báo mất kết nối tạo 2 lần, tổng hợp chạy chồng
File đính kèm lưu ổ đĩa cục bộ (`Storage:Root`)	Ảnh tải lên máy A, máy B không đọc được

Thành phần chỉ có ở một máy	Hậu quả khi có 2 instance
`BackfillTracker` và `IMemoryCache` của thống kê	Máy B tiếp tục trả số liệu cũ đã cache sau khi máy A tính lại

2.1 Điểm nghẽn ngay cả khi chỉ có một instance

Giả định theo `contract/realtime.md`: telemetry ≈ 5 s/lần, pin ≈ 20 s/lần → mỗi thiết bị ≈ 0,25 bản tin/giây, ≈ 21.600 dòng dữ liệu thô/ngày.

- **Job tổng hợp (lớn nhất):** mỗi 30 s quét 35 phút dữ liệu thô của **mọi** thiết bị và tính lại ≈ 4 ngày `rollup_15m` của mọi thiết bị. 1.000 thiết bị ≈ 0,4 + 0,4 triệu dòng/30 s; 10.000 thiết bị ≈ 4 + 4 triệu dòng/30 s.
- **Dung lượng:** ước 4–5 MB/thiết bị/ngày dữ liệu thô; giữ 90 ngày ≈ 400 GB cho mỗi 1.000 thiết bị (ước lượng theo số cột, cần đo).
- **Tra serial** bằng `upper(serial) = upper(@serial)` ở 6 chỗ (có kiểm tra quyền mỗi request) mà không có index tương ứng → quét toàn bảng.
- **MQTT** `CleanSession`: backend khởi động lại / deploy thì broker không giữ bản tin → **mất dữ liệu** trong thời gian đó.
- **Ghi DB ngay trong handler MQTT** (status, cảnh báo, config/state): DB chậm thì nhận bản tin chậm theo.
- `/healthz` **không kiểm tra DB**, không có metrics: không biết trước khi chạm trần.

3. Mục tiêu và năng lực sau từng giai đoạn

Các con số là **ước lượng bậc độ lớn**, phải chốt bằng bài thử tải A0.

Giai đoạn	Số thiết bị (ước)	Người dùng app	Dự phòng / deploy
Hiện tại	~1.000–2.000 (job tổng hợp, dung lượng)	Vài nghìn kết nối WebSocket	Một điểm hỏng; deploy mất dữ liệu
A · Nhanh	~5.000 trên một máy (~1.250 bản tin/giây)	Như trên	Deploy không mất dữ liệu, vẫn gián đoạn ngắn
B · Trung bình	~5.000–10.000 (nhận dữ liệu vẫn một máy)	Tăng theo số máy API, cỡ chục nghìn đồng thời	2+ instance, deploy cuốn chiếu không gián đoạn
C · Lớn	~50.000–100.000; trần là tốc độ ghi DB (100.000 thiết bị ≈ 25.000 dòng/giây) và cụm broker	Như trên	Hỏng một worker thì broker chia lại thiết bị

4. Giai đoạn A — Nhanh (vẫn một instance)

Mục tiêu: nâng trần lên ~5.000 thiết bị, không mất dữ liệu khi deploy, nhìn thấy được hệ thống đang ở đâu so với trần. Rủi ro thấp, không đổi API.

A0 · Bài thử tải và số đo nền

Mục	Nội dung

Mục	Nội dung
Mục tiêu	Có công cụ giả lập N thiết bị và bộ số đo nền trước khi sửa gì.
Vì sao	Mọi con số ở mục 3 đang là ước lượng; không có số nền thì không chứng minh được nâng cấp có tác dụng.
Việc cần làm	• Tạo `backend/tools/GgPower.LoadGen` (console .NET): giả lập N thiết bị gửi `up/status` (1 lần), `up/telemetry` mỗi 5 s, `up/battery` mỗi 20 s qua MQTT, lệch pha ngẫu nhiên; tham số `--devices`, `--minutes`, `--broker`. • Broker thử: Mosquitto trong Docker với cấu hình riêng cho test (cho phép tài khoản thử). • Đo: bản tin nhận/giây, `rows_dropped`, `last_batch_ms` (từ `/healthz`), thời gian một lần tổng hợp (log `RollupService`), CPU/RAM của API và Postgres, dung lượng `telemetry_raw` sau 30 phút. • Chạy 1.000 / 2.000 / 5.000 thiết bị, ghi bảng kết quả vào `backend/docs/LOADTEST.md`.
File liên quan	`backend/tools/GgPower.LoadGen/` (mới), `backend/docs/LOADTEST.md` (mới), `backend/README.md` (cách chạy)
Xong khi	• Có bảng số nền cho 3 mức tải • Công cụ chạy lại được bằng một lệnh
Rủi ro / lưu ý	Máy dev có thể không đủ mạnh cho 5.000 thiết bị: ghi rõ cấu hình máy cạnh kết quả.

A1 · Index cho serial không phân biệt hoa thường

Mục	Nội dung
Mục tiêu	Mọi truy vấn `upper(serial) = upper(@serial)` dùng index.
Vì sao	Kiểm tra quyền chạy mỗi request; quét toàn bảng tăng tuyến tính theo số thiết bị.
Việc cần làm	• Migration `003_scale_indexes.sql`: index biểu thức `upper(serial)` trên `project_devices`, `config_allowlist`, `devices`; `config_commands (upper(serial), created_at DESC)`. • Rà lại 6 truy vấn dùng `upper(...)` để khớp đúng biểu thức của index. • Test: `EXPLAIN` truy vấn kiểm tra quyền thiết bị không còn `Seq Scan` (với bảng có dữ liệu mẫu).
File liên quan	`src/GgPower.Infrastructure/Sql/003_*.sql`, `Api/Common.cs`, `Infrastructure/Commands.cs`, `Api/Endpoints/*`
Xong khi	• Test EXPLAIN xanh • `dotnet test` xanh

A2 · Health check liveness / readiness

Mục	Nội dung
Mục tiêu	Load balancer / Docker biết chính xác instance nào sẵn sàng phục vụ.

Mục	Nội dung
Vì sao	Điều kiện bắt buộc trước khi chạy nhiều instance (giai đoạn B) và để deploy cuốn chiếu.
Việc cần làm	• `GET /healthz/live`: tiến trình còn chạy (luôn 200). • `GET /healthz/ready`: 200 khi `SELECT 1` tới DB ≤ 2 s, MQTT đã kết nối (nếu instance có vai trò nhận dữ liệu), hàng đợi ghi < 80 % dung lượng; ngược lại 503 kèm lý do. • Giữ `/healthz` cũ (chi tiết số liệu) để không phá công cụ đang dùng. • `deploy/docker-compose.yml`: healthcheck của `api` gọi `/healthz/ready`.
File liên quan	`src/GgPower.Api/Program.cs`, `deploy/docker-compose.yml`
Xong khi	• Test: ready = 200 khi bình thường; = 503 khi giả lập DB lỗi (connection string sai trong test riêng)

A3 · Metrics (OpenTelemetry → Prometheus)

Mục	Nội dung
Mục tiêu	Đo liên tục: tốc độ nhận, độ trễ ghi, thời gian tổng hợp, số kết nối, lỗi lệnh.
Vì sao	Biết trước khi chạm trần; là thước đo cho mọi bước sau.
Việc cần làm	• Thêm OpenTelemetry + exporter Prometheus; endpoint `/metrics` chỉ mở trong mạng nội bộ (Caddy không chuyển ra ngoài). • Meter `GgPower`: `ingest.received`, `ingest.dropped`, `ingest.queue_length`, `writer.rows`, `writer.batch_ms`, `rollup.duration_ms`, `rollup.rows`, `realtime.connections`, `commands.submitted/applied/failed`, kèm metrics HTTP chuẩn của ASP.NET Core. • Ghi tên metric và ý nghĩa vào `backend/docs/METRICS.md`.
File liên quan	`GgPower.Api.csproj`, `Program.cs`, `Infrastructure/Ingest.cs`, `Infrastructure/Commands.cs`, `deploy/Caddyfile`
Xong khi	• `/metrics` trả các metric trên • Test gọi `/metrics` thấy `ingest_received_total`
Cần người dùng quyết	Thêm package NuGet `OpenTelemetry.Extensions.Hosting`, `OpenTelemetry.Instrumentation.AspNetCore`, `OpenTelemetry.Exporter.Prometheus.AspNetCore` (đề xuất) hay chỉ dùng `System.Diagnostics.Metrics` + endpoint tự viết.

A4 · Tổng hợp tăng dần

Mục	Nội dung
Mục tiêu	Chi phí job tổng hợp tỉ lệ với **dữ liệu mới**, không tỉ lệ với tổng số thiết bị × 4 ngày.
Vì sao	Đây là điểm nghẽn lớn nhất của database (mục 2.1).
Việc cần làm	• `BatchWriter` ghi lại tập "bẩn": serial → thời điểm mẫu sớm nhất

Mục	Nội dung
	vừa ghi (gồm cả mẫu đến trễ mà `BackfillTracker` đang xử lý). • `RollupService.RunOnceAsync` lấy tập bẩn: `Rollup15` chỉ cho các serial đó, từ khung 15 phút chứa mẫu sớm nhất; tổng hợp ngày chỉ cho các ngày địa phương bị chạm; `RefreshChargedTodayAsync` chỉ cho các serial đó. • Giữ một lượt "quét an toàn" toàn bộ mỗi 6 giờ (phòng sót), chạy ngoài giờ cao điểm. • Giữ nguyên SQL tính năng lượng (tích phân theo `dt`) để số liệu không đổi.
File liên quan	`src/GgPower.Infrastructure/Ingest.cs` (`BatchWriter`, `RollupService`, `BackfillTracker`)
Xong khi	• Mọi test thống kê hiện có xanh, số liệu không đổi • Test mới: thiết bị không có mẫu mới không bị tính lại • Chạy lại A0: thời gian một lần tổng hợp ở 5.000 thiết bị giảm rõ rệt (ghi vào LOADTEST.md)
Rủi ro / lưu ý	Mẫu đến trễ qua nửa đêm: ngày hôm trước phải được đánh dấu bẩn.

A5 · MQTT persistent session

Mục	Nội dung
Mục tiêu	Backend khởi động lại / deploy không mất bản tin.
Vì sao	Hiện `CleanSession` nên broker bỏ bản tin trong lúc backend tắt.
Việc cần làm	• `MqttLink`: `CleanSession = false` (hoặc MQTT 5 với `SessionExpiryInterval`), client id cố định theo cấu hình (Mosquitto đang lấy client id = tên đăng nhập). • Mosquitto: tăng `max_queued_messages` cho tài khoản backend (mặc định 1.000 quá nhỏ: 5.000 thiết bị × 0,25 bản tin/giây × 60 s = 75.000 bản tin/phút tắt máy); tính lượng RAM/đĩa của broker. • Kiểm tra trùng lặp khi broker gửi lại (QoS 1 "ít nhất một lần"): dữ liệu thô đã bỏ qua trùng; cảnh báo theo `alarm_id`; status idempotent. Viết test cho từng loại. • Ghi hướng dẫn đổi cấu hình broker vào `backend/docs/DEPLOY.md`.
File liên quan	`Infrastructure/Mqtt.cs`, `deploy/mqtt/config/mosquitto.conf`, `backend/docs/DEPLOY.md`
Xong khi	• Thử thủ công với Mosquitto Docker: tắt API 60 s trong lúc LoadGen chạy → bật lại → không thiếu mẫu (đếm `telemetry_raw`) • Test trùng lặp xanh
Cần người dùng quyết	Mức `max_queued_messages` / thời gian giữ phiên (đề xuất: đủ cho 10 phút tắt máy ở quy mô dự kiến).

A6 · Ghi status / cảnh báo / config-state qua hàng đợi

Mục	Nội dung
Mục tiêu	Handler MQTT chỉ cập nhật RAM và xếp hàng; việc ghi DB làm ở nền.

Mục	Nội dung
Vì sao	DB chậm không làm nghẽn việc nhận bản tin (giống telemetry đang làm).
Việc cần làm	• Hàng đợi có giới hạn cho sự kiện thiết bị; một consumer nền ghi DB, **giữ thứ tự theo từng thiết bị**. • Đầy hàng đợi: ghi log + metric, không chặn handler (status cũ có thể bỏ, cảnh báo thì không được bỏ → dung lượng riêng). • Metric thời gian xử lý handler.
File liên quan	`Infrastructure/Ingest.cs`, `Infrastructure/Notifications.cs`
Xong khi	• Test cảnh báo / status hiện có xanh • Test: DB chậm giả lập không làm tăng thời gian handler

A7 · Đo lại và chốt giai đoạn A

Mục	Nội dung
Mục tiêu	Chứng minh giai đoạn A đạt mục tiêu.
Vì sao	Để quyết định có cần làm tiếp B, C ngay hay chưa.
Việc cần làm	• Chạy lại A0 ở 1.000 / 2.000 / 5.000 thiết bị • So sánh với số nền trong `LOADTEST.md` • Cập nhật bảng năng lực mục 3 trong PROGRESS.md bằng số thật
File liên quan	`backend/docs/LOADTEST.md`, `PROGRESS.md`
Xong khi	• Báo cáo so sánh trước / sau gửi người dùng

5. Giai đoạn B — Trung bình (nhiều instance API)

Mục tiêu: chạy 2+ instance API sau load balancer, deploy cuốn chiếu không gián đoạn. Phần nhận MQTT vẫn chỉ một instance (cờ vai trò ở B7). Mọi phụ thuộc mới đi qua **interface** có bản trong RAM mặc định, để test và môi trường một máy không cần Redis.

B0 · Chốt hạ tầng chia sẻ

Mục	Nội dung
Mục tiêu	Chọn Redis và nơi lưu file.
Vì sao	Trạng thái dùng chung giữa các instance cần một nơi chung, nhanh.
Việc cần làm	• Hỏi người dùng (popup) rồi ghi quyết định vào PROGRESS.md mục 2.
File liên quan	—
Xong khi	• Quyết định đã ghi
Cần người dùng quyết	Redis: container `redis:7` trong compose (đề xuất) hay dịch vụ quản lý. Object storage: MinIO tự host (đề xuất) hay S3 trên cloud. Package: `StackExchange.Redis`, `AWSSDK.S3`.

B1 · Bầu leader cho job định kỳ

Mục	Nội dung
Mục tiêu	Mỗi job định kỳ chỉ chạy ở đúng một instance.
Vì sao	Tránh báo cáo ngày gửi 2 lần, cảnh báo offline tạo 2 lần, tổng hợp chạy chồng.
Việc cần làm	• `JobLeader`: giữ `pg_try_advisory_lock(<khoá theo job>)` trên một kết nối riêng; mất kết nối thì mất quyền, instance khác thay. • Áp cho: `RollupService` (gồm dọn partition, audit, OTP, session), `NotificationService` (báo cáo ngày), `LiveMonitor` (cảnh báo offline), nạp lại TOU 00:00. • Metric `jobs.leader{job}` = 1 / 0.
File liên quan	`Infrastructure/` (lớp mới `JobLeader`), `Ingest.cs`, `Notifications.cs`, `Commands.cs`
Xong khi	• Test: hai `RollupService` cùng DB → chỉ một cái chạy; leader dừng → cái kia nhận trong ≤ 1 chu kỳ

B2 · Tạo cảnh báo bằng upsert

Mục	Nội dung
Mục tiêu	Hai tiến trình cùng lúc không tạo trùng cảnh báo.
Vì sao	Khi có nhiều instance, `AlertStore.RaiseAsync` kiểu "đọc rồi ghi" có thể tạo 2 dòng.
Việc cần làm	• Migration: gộp các cảnh báo mở bị trùng (nếu có), đổi `alerts_open_ix` thành **unique** `(serial, code) WHERE ended_at IS NULL`. • `RaiseAsync`: `INSERT … ON CONFLICT DO NOTHING RETURNING` trong transaction; giữ quy tắc mở lại sau `AlertReopenMinutes`.
File liên quan	`Infrastructure/Ingest.cs` (`AlertStore`), `Sql/004_*.sql`
Xong khi	• Test chạy song song 20 lần raise cùng lỗi → đúng 1 cảnh báo

B3 · Tách interface cho trạng thái dùng chung

Mục	Nội dung
Mục tiêu	Mọi trạng thái "chỉ có ở một máy" đi qua interface; bản mặc định vẫn trong RAM (hành vi không đổi).
Vì sao	Bước chuẩn bị để thay bằng Redis ở B4 mà không sửa logic nghiệp vụ.
Việc cần làm	• `ITicketStore` (vé realtime), `ICommandCoordinator` (khoá lệnh theo thiết bị, giới hạn tốc độ, chờ ack), `ICacheInvalidation` (thay `BackfillTracker.Version` cho cache), `IRealtimeBus` (phát frame `device`), `IFileStore` (file đính kèm). • Bản trong RAM bọc code hiện tại; DI chọn theo cấu hình `Shared:Provider = memory | redis`.
File liên quan	`Api/Endpoints/DataEndpoints.cs` (`RealtimeTickets`),

Mục	Nội dung
	`Infrastructure/Commands.cs`, `Infrastructure/LiveState.cs` (`RealtimeHub`), `Api/Common.cs` (file), `Program.cs`
Xong khi	• `dotnet test` xanh, không đổi hành vi

B4 · Bản Redis cho vé, khoá lệnh, cache

Mục	Nội dung
Mục tiêu	Vé realtime, khoá lệnh, giới hạn tốc độ và việc xoá cache dùng chung giữa các instance.
Vì sao	Sửa các lỗi "vé tạo ở A dùng ở B", "hai lệnh cùng lúc tới một thiết bị", "B trả số cũ".
Việc cần làm	• Vé: `SET ticket:{id} … EX 30`, dùng một lần bằng `GETDEL`. • Khoá lệnh: `SET cmdlock:{serial} {cmdId} NX PX <thời hạn>`; giới hạn tốc độ: `SET cmdlast:{serial} NX PX <MinIntervalSeconds>`. • Xoá cache: `INCR backfill:{serial}` + pub/sub để các instance bỏ cache thống kê đã đóng. • Test tích hợp với Redis trong Docker (bỏ qua nếu không có Redis, giống cách đang làm với Postgres).
File liên quan	`Infrastructure/Shared/Redis*.cs` (mới), `Program.cs`, `GgPower.Tests`
Xong khi	• Test: vé tạo ở host A dùng được ở host B; hai host cùng gửi lệnh tới một thiết bị → một cái nhận `COMMAND_IN_PROGRESS`

B5 · Phát realtime và ack lệnh qua Redis pub/sub

Mục	Nội dung
Mục tiêu	App nối vào instance nào cũng nhận frame của mọi thiết bị; ack lệnh về instance nào cũng tới đúng người chờ.
Vì sao	Sửa lỗi WebSocket và lệnh "không rõ" khi có nhiều instance.
Việc cần làm	• Instance nhận dữ liệu publish frame `device` (byte đã serialize) lên kênh `rt:{serial}` (hoặc một kênh chung có serial trong frame); mỗi instance API subscribe và đẩy cho WebSocket cục bộ. • Ack lệnh: instance nhận `up/ack` publish `ack:{cmdId}`; instance đang chờ lệnh đó nhận và hoàn tất. Trạng thái lệnh trong DB vẫn là nguồn sự thật. • Giữ hàng đợi 32 frame / kết nối như hiện tại.
File liên quan	`Infrastructure/LiveState.cs`, `Infrastructure/Commands.cs`, `Api/Endpoints/DataEndpoints.cs`
Xong khi	• Test hai host: WebSocket ở B nhận frame của dữ liệu đưa vào A; lệnh gửi ở B, ack vào A → B trả `applied`

B6 · File đính kèm lên object storage

Mục	Nội dung
Mục tiêu	File đọc được từ mọi instance.
Vì sao	Hiện lưu ổ đĩa cục bộ của một máy.
Việc cần làm	• `S3FileStore` (MinIO / S3), giữ `LocalFileStore` cho dev. • Script chuyển file cũ từ `deploy/data/storage` lên bucket; ghi vào `DEPLOY.md`. • URL tải: proxy qua API (giữ kiểm tra quyền) hoặc presigned URL ngắn hạn.
File liên quan	`Api/Common.cs`, `Infrastructure/Shared/`, `deploy/docker-compose.yml` (MinIO)
Xong khi	• Test tải lên / tải về với MinIO Docker • Ticket có ảnh tạo ở A xem được ở B

B7 · Chạy 2 instance và deploy cuốn chiếu

Mục	Nội dung
Mục tiêu	Hai instance API sau Caddy; một instance có vai trò nhận dữ liệu.
Vì sao	Đích của giai đoạn B: không còn một điểm hỏng ở phía API, deploy không gián đoạn.
Việc cần làm	• Cấu hình vai trò `Roles: api | ingest | jobs` (cùng một image). Instance 1: `api,ingest,jobs`; instance 2: `api` (jobs vẫn có leader ở B1 nên bật cả hai cũng an toàn). • Caddy cân bằng tải 2 upstream, health check `/healthz/ready`; WebSocket không cần sticky nhờ B4–B5. • Viết quy trình deploy cuốn chiếu vào `DEPLOY.md`. • Chạy lại A0 + kịch bản: tắt instance 2 giữa chừng, app không lỗi.
File liên quan	`Program.cs`, `deploy/docker-compose.yml`, `deploy/Caddyfile`, `backend/docs/DEPLOY.md`
Xong khi	• Compose 2 instance chạy được cục bộ • Kết quả đo ghi vào LOADTEST.md

6. Giai đoạn C — Lớn (tách nhận dữ liệu, ~50.000+ thiết bị)

Chỉ làm khi số thiết bị thực tế tiến tới vài nghìn hoặc số đo A0 cho thấy một máy nhận dữ liệu sắp đầy. Mỗi bước ở đây có chi phí hạ tầng; luôn hỏi người dùng trước.

C1 · Tách vai trò thành service riêng

Mục	Nội dung
Mục tiêu	Worker nhận dữ liệu, API và job chạy thành các service độc lập (cùng image).
Vì sao	Co giãn từng phần theo tải riêng của nó.
Việc cần làm	• Dùng cờ vai trò từ B7; compose có `api` (N bản), `ingest` (1 bản),

Mục	Nội dung
	`jobs` (1 bản + leader) • Health check riêng cho từng vai trò
File liên quan	`Program.cs`, `deploy/docker-compose.yml`
Xong khi	• Ba service chạy được, test tích hợp xanh

C2 · Trạng thái realtime lên Redis

Mục	Nội dung
Mục tiêu	API đọc trạng thái thiết bị từ Redis, không phụ thuộc worker nào đang xử lý thiết bị.
Vì sao	Điều kiện để có nhiều worker nhận dữ liệu.
Việc cần làm	• Worker ghi ảnh chụp gọn `dev:{serial}` (hash / JSON) mỗi khi cập nhật, cùng lúc publish frame (B5). • API đọc Redis, có cache cục bộ vài trăm ms; khởi động worker nạp lại từ Redis + DB. • Trạng thái tích luỹ theo thiết bị (`BattFromPvKwh`, khoảng `dt` giữa hai mẫu) vẫn do **đúng một worker** giữ — xem C3.
File liên quan	`Infrastructure/LiveState.cs`, `Api/Endpoints/*`
Xong khi	• Test: API không có ingest vẫn trả `/overview` đúng từ Redis

C3 · Nhiều worker nhận dữ liệu (shared subscription có gắn thiết bị)

Mục	Nội dung
Mục tiêu	N worker chia nhau thiết bị, **mỗi thiết bị luôn về cùng một worker**.
Vì sao	Mosquitto chia `$share` kiểu luân phiên, không gắn thiết bị → hỏng phép tích phân năng lượng theo thời gian.
Việc cần làm	• Chuyển broker sang EMQX (hỗ trợ chiến lược `hash_topic` / `hash_clientid` cho shared subscription, chạy cụm). • Worker đăng ký `$share/ingest/solar/+/inverter/+/up/#`; thêm / bớt worker thì broker chia lại. • Chuyển dữ liệu `passwd`/`acl` của Mosquitto sang EMQX; giữ nguyên topic để firmware không phải đổi.
File liên quan	`deploy/` (EMQX), `Infrastructure/Mqtt.cs`, `backend/docs/DEPLOY.md`
Xong khi	• LoadGen 20.000+ thiết bị với 2–3 worker; tắt một worker, thiết bị chuyển sang worker khác trong vài giây
Cần người dùng quyết	Đổi broker sang EMQX (đề xuất) hay giữ Mosquitto và chỉ dùng một worker.

C4 · TimescaleDB cho dữ liệu thô

Mục	Nội dung

Mục	Nội dung
Mục tiêu	Giảm dung lượng 10–20 lần, truy vấn theo thời gian nhanh hơn.
Vì sao	Dung lượng dữ liệu thô là trần thứ hai sau tốc độ ghi (≈ 400 GB / 1.000 thiết bị / 90 ngày).
Việc cần làm	• Image Postgres + extension TimescaleDB; chuyển `telemetry_raw`, `battery_raw` thành hypertable (thay partition tự quản ở `PartitionMaintenance`). • Chính sách nén sau 2 ngày, chính sách xoá theo `RawRetentionDays`. • **Giữ job tổng hợp tăng dần của A4**: phép tính năng lượng dùng `LAG` (hàm cửa sổ) mà continuous aggregate không hỗ trợ. • Kịch bản chuyển dữ liệu cũ, đo thời gian trên bản sao.
File liên quan	`Sql/005_timescale.sql`, `Infrastructure/Ingest.cs` (`PartitionMaintenance`), `deploy/docker-compose.yml`
Xong khi	• Test thống kê xanh trên Timescale • Đo dung lượng trước / sau ghi vào LOADTEST.md
Cần người dùng quyết	Dùng TimescaleDB (giấy phép Timescale License, bản community) hay giữ Postgres thường và giảm thời gian giữ dữ liệu thô.

C5 · Xác thực MQTT theo từng thiết bị

Mục	Nội dung
Mục tiêu	Mỗi datalogger có thông tin đăng nhập riêng, chỉ được ghi/đọc topic của chính nó.
Vì sao	Quy mô lớn thì danh sách mật khẩu tĩnh không quản lý được và rủi ro bảo mật cao.
Việc cần làm	• EMQX xác thực qua HTTP tới backend hoặc bảng DB; ACL theo serial • Quy trình cấp / thu hồi thông tin đăng nhập khi gắn / gỡ thiết bị
File liên quan	`Api/Endpoints/` (endpoint xác thực nội bộ), `deploy/`
Xong khi	• Thiết bị A không publish được vào topic của thiết bị B (test với EMQX Docker)
Cần người dùng quyết	Cách cấp thông tin đăng nhập cho datalogger (cần phối hợp firmware).

C6 · Vận hành

Mục	Nội dung
Mục tiêu	Build, kiểm thử, sao lưu và giám sát tự động.
Vì sao	Nhiều service hơn thì không thể làm tay.
Việc cần làm	• CI (GitHub Actions): build, `dotnet test` với Postgres service, build và đẩy image. • Thay CentOS 7 (hết hỗ trợ) bằng Rocky/Alma 9 hoặc Ubuntu 24.04

Mục	Nội dung
	— viết hướng dẫn, không tự đổi máy chủ. • Sao lưu Postgres ra ngoài máy (WAL-G / pgBackRest lên object storage), có bài kiểm tra khôi phục. • Dashboard Grafana từ metrics A3 + cảnh báo (hàng đợi đầy, mất MQTT, ready = 503).
File liên quan	`.github/workflows/`, `backend/docs/DEPLOY.md`, `deploy/backup.sh`
Xong khi	• CI xanh trên PR • Hướng dẫn khôi phục đã chạy thử
Cần người dùng quyết	Nền tảng CI, hệ điều hành máy chủ, nơi lưu backup.

7. Danh sách quyết định cần người dùng chốt

Hỏi đúng lúc tới bước tương ứng, bằng popup, phương án đề xuất đứng đầu.

#	Bước	Câu hỏi	Đề xuất
D1	A3	Package OpenTelemetry hay metrics tự viết	OpenTelemetry + Prometheus exporter
D2	A5	Hàng đợi broker giữ được bao lâu khi backend tắt	Đủ 10 phút ở quy mô dự kiến
D3	B0	Redis tự host hay dịch vụ quản lý	Container `redis:7` trong compose
D4	B0	Object storage	MinIO tự host
D5	C3	Đổi broker sang EMQX	Có, khi cần hơn một worker
D6	C4	TimescaleDB hay giảm thời gian giữ dữ liệu thô	TimescaleDB community
D7	C5	Cấp thông tin đăng nhập MQTT cho datalogger	Bàn với nhóm firmware
D8	C6	CI, hệ điều hành, nơi backup	GitHub Actions, Ubuntu 24.04, object storage ngoài máy

8. Bảng theo dõi (sao sang PROGRESS.md mục 4.4)

Bước	Nội dung	Trạng thái	Commit
A0	Bài thử tải, số nền	Chưa làm	
A1	Index serial	Chưa làm	
A2	Health live/ready	Chưa làm	
A3	Metrics	Chưa làm	
A4	Tổng hợp tăng dần	Chưa làm	

Bước	Nội dung	Trạng thái	Commit
A5	MQTT persistent session	Chưa làm	
A6	Ghi sự kiện qua hàng đợi	Chưa làm	
A7	Đo lại, chốt A	Chưa làm	
B0–B7	Nhiều instance API	Chưa làm	
C1–C6	Tách nhận dữ liệu, quy mô lớn	Chưa làm	

9. Rủi ro chung và cách lùi lại

- Mỗi bước một commit, có thể `git revert` độc lập; migration SQL chỉ **thêm** (index, cột, ràng buộc) trừ khi bước ghi rõ.
- Bản trong RAM của các interface (B3) luôn được giữ: tắt Redis bằng cấu hình `Shared:Provider = memory` để quay về một instance.
- Đổi broker (C3) và Timescale (C4) làm trên môi trường thử với bản sao dữ liệu trước; có kịch bản quay lại.
- Số liệu thống kê phải giống hệt trước / sau mỗi bước chạm tới tổng hợp (A4, C4): test so sánh trên cùng bộ mẫu.